law-bench: An Evaluation Loop and Self-Healing Generation System for Chinese Contract Drafting

Authors: Chengsi Shi[1], Fenglan Lan[2]

Affiliations: [to be completed: department, institution, city, postal code; corresponding-author e-mail]

Abstract

Drafting Chinese contracts depends heavily on legal norms: a single missing mandatory clause or a mis-cited statutory basis can invalidate the agreed terms. Large language models can already produce fluent contract text, yet whether the output satisfies each normative requirement point by point lacks a reproducible measure; whether a prompt or workflow change actually improves quality cannot be quantified; and quality regressions go unnoticed. This paper presents law-bench, an evaluation loop and self-healing generation system for Chinese contract drafting. The system organizes a clause corpus by legal-topic tags and assembles contracts with slot filling; for each generated contract, multiple LLM judges independently rule on every review criterion, and an all-pass binary aggregation emits the quality signal — a contract counts as passing only if every criterion passes, so that a high average can no longer mask a shortboard. The signal further triggers clause-level self-healing iterations that fix defects by merging rather than replacing, preserving what earlier rounds got right. On 43 Chinese contract types covering the typical contracts of the Civil Code and common atypical contracts, the system brings 39 types to all-criteria pass (18/19 typical, 21/24 atypical); the 4 failing types split into two structural boundaries: a clause-quality wall, where improvement still cannot satisfy all judges simultaneously, and a judge-reliability ceiling, where verdicts oscillate across rounds. The experiments also show that upstream rate limits are metered per account rather than per model, so any pass-rate-scale conclusion must state its concurrency configuration. The evaluation loop makes contract quality measurable, iterable, and regression-detectable; a blind human-annotation pilot (126 contract–criterion pairs, double-annotated) measures inter-annotator agreement at 85.7% ($\kappa = 0.522$) but human–judge agreement at only 36.5%, showing that judge verdicts are systematically stricter than human

blind judgment — the system therefore uses the judge signal only to drive loop termination, embeds human review inside the loop, and never treats machine self-evaluation as gold data. A multi-run stability report remains future work.

Keywords: large language models; Chinese contract drafting; automatic evaluation; LLM-as-a-judge; self-healing iteration

1 Introduction

Contracts are texts under strong legal constraint: their validity depends on satisfying legal requirements point by point, and one missing mandatory clause or one mis-cited statute can invalidate the agreed terms. Drafting is thus language production overlaid with norm verification, and it underpins the batch production of standardized documents. Large language models already generate structurally complete, fluent Chinese contracts, but a systematic gap separates "reads like a contract" from "holds up legally": characterizations of legal hallucination show that models fabricate statutes and precedents as a stable phenomenon tied to how questions are posed [1]. Such defects lurk in plausible-looking, groundless clauses that manual review can hardly check exhaustively. Automating contract drafting must therefore also answer "was it written correctly, and how do we know".

Existing work on legal text automation follows three routes. The domain-adaptation route continues training or fine-tuning models on legal corpora to acquire jurisdictional phrasing [5]. The task-application route applies models to concrete text processing: clause-level contract analysis is already automated [6], and legal question generation has been systematically examined [7]. The benchmark route carries the benchmarking tradition into law, decomposing legal reasoning into task collections [3], with comprehensive Chinese-jurisdiction benchmarks spanning multiple cognitive levels [4]. All three routes measure task accuracy, judgment of existing texts, or question answering — none measures "how many normative requirements a contract satisfies"; the shared gap is the absence of a reproducible measure for drafting quality. When output is handed to drafters, quality still rests on manual review, change effects cannot be quantified, and regressions pass unnoticed [2].

Wiring evaluation into generation requires a reproducible signal source and a usable grading mechanism. The benchmarking tradition shows that decomposing quality demands

into controlled tasks is the precondition for comparable results [3], and the comprehensive Chinese benchmark extends this to multi-level cognitive tasks [4]; yet their published task compositions center on question answering, reasoning, and information extraction — generating a complete text that satisfies normative requirements point by point is not a benchmark mainstay. On the grading side, LLMs have been shown to work as evaluators aligned reasonably well with human judgment [8], but with position and verbosity biases [9], validity boundaries that demand comparison against humans [10], judge-model differences large enough to flip conclusions [12], and reliability becoming the pipeline bottleneck under strict verdicts [15]. Clause-granular automatic judgment has been validated as feasible [6], which lets the review unit be refined accordingly. Existing work mostly treats evaluation as one-off post-generation checking rather than a driving signal; a closed loop that takes all-criteria-pass of per-criterion review as the criterion and drives generation in reverse remains vacant. The judges' limitations are both a risk to face and a design constraint of this paper: conclusions must come from multiple judges with a confidence mechanism.

This paper presents law-bench, an evaluation loop and self-healing generation system for Chinese contract drafting, turning evaluation from post-hoc inspection into an intrinsic driver of generation so that quality becomes measurable, iterable, and regression-detectable. The system assembles contracts from a clause corpus organized by legal-topic tags, adopts all-pass binary scoring over multi-judge per-criterion review as the quality criterion, and triggers clause-level self-healing iterations. Its contributions are fourfold: an all-pass binary evaluation protocol under which a contract counts as passing only when every review criterion passes, giving prompt- and version-level comparisons a regression-testable baseline; a self-healing mechanism that triggers improvement by multi-judge confidence threshold and updates clauses by merging rather than replacing; an evaluation set of 43 Chinese contract types covering Civil Code typical contracts and atypical contracts, with a reproducible pipeline; and a taxonomy distinguishing clause-quality structural failures from judge-reliability measurement failures.

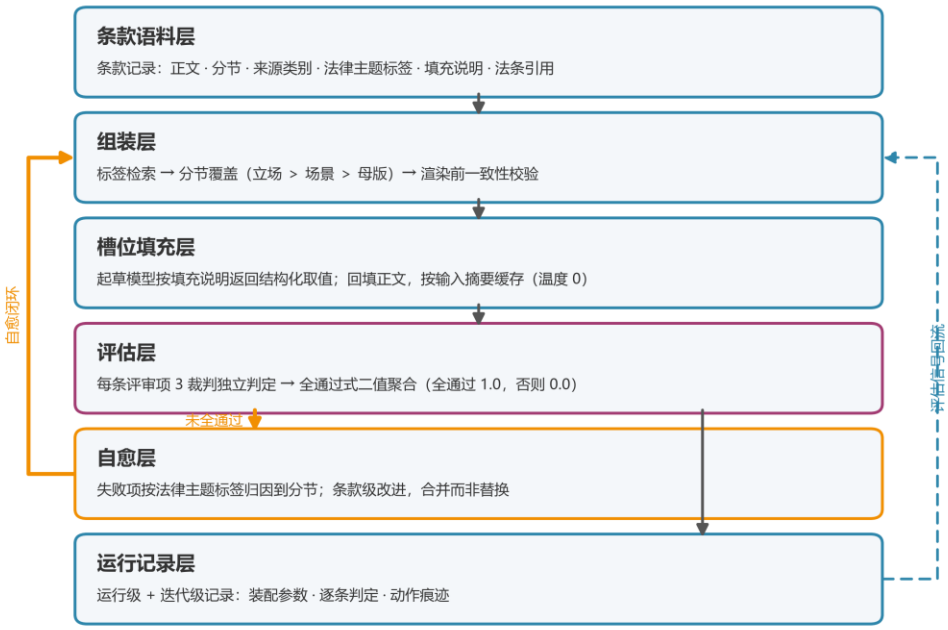
The remainder is organized as follows. Section 2 presents the overall system design, covering the responsibilities and data flow of the corpus, assembly, evaluation, and self-healing layers. Section 3 details the key methods: the evaluation protocol, the self-healing mechanism, and tag-driven assembly. Section 4 reports the evaluation results, threshold

sensitivity, failure cases, judge-validity pilot, and engineering constraints under concurrency. Section 5 discusses the validity of the binary signal, the reliability boundary of measurement, and limitations. Section 6 concludes with prospects.

2 Overall System Design

2.1 Architecture

The clause corpus is assembled by tags into a body with placeholders; a drafting model instantiates the placeholders per fill instructions; the text goes to multiple judges for per-criterion verdicts; failing items return to assembly to drive the next generation round — the evaluation signal becomes an input to the next round, and Figure 1 shows the inputs, outputs, and feedback edges.



评估信号经自愈层回流至组装层，构成起草—评估—改进闭环；运行记录层留存逐条判定与动作痕迹，供回归比较与追溯。

Figure 1 Overall architecture and data flow of law-bench

The corpus layer takes extracted texts and manually authored clauses as input, normalizes them into canonical sections, unifies placeholder names, and stamps source and topic tags; it outputs master, scenario, and stance records for assembly-time retrieval and override.

The assembly layer takes contract type, scenario and stance tags, and approved clauses; section-level override resolves stance over scenario, scenario over master, with master as fallback, one body per section, and section completeness validated before rendering. It outputs the body, the placeholder list, fill instructions, and statutory citations; the policy makes results reproducible.

The filling layer takes the assembled result and drafting requirements; the drafting model returns placeholder values per fill instructions and substitutes them, outputting the contract text and the list of unresolved placeholders. Results are cached by input digest at temperature zero, hence reproducible.

The evaluation layer takes the filled text and the rubric; each review criterion is ruled on independently by multiple judge models as a binary verdict, then aggregated all-pass into the quality signal, outputting the all-pass flag, the pass count, and per-criterion reasoning. A single missing mandatory clause negates the whole.

The self-healing layer takes failing items, current-round parameters, and history; after attribution it produces two kinds of actions within a run — excluding stance clauses of a designated position and recommending retirement of specific clauses; exclusions never write back to the shared clause store. The run-record layer stores parameters, texts, and verdicts at run and iteration levels for cross-round comparison.

2.2 Data Model

The contract types are 19 typical nominate contracts of the Civil Code plus 24 atypical contracts, 43 in total — the unified denominator for assembly and evaluation; the remaining types stay in the clause and template stores without entering the evaluation set, so all per-type-count conclusions use 43 as denominator.

A clause record carries the body, section, source type, topic tags, fill instructions, statutory citations, and per-dimension approval status; placeholder names are normalized on

write and content digests derived for deduplication. A stance clause enters the assembly candidates only after approval on every tag dimension.

Review criteria are organized per rubric — one ordered item set per contract type, each with a stable identifier, the object under check, and the pass standard. Evaluation results are persisted as text snapshots, so later rubric edits never alter historical verdicts. Run records are organized at two levels: run level stores assembly parameters, fill values, and the best-round text linked to its evaluation; iteration level stores the round number and action traces.

2.3 Three Pipelines

The standard self-healing pipeline transitions through generate, fill, evaluate, check, improve, and store: on all-pass or round limit it moves to store, otherwise to improve and back to generate. Inputs are contract type, stance tag, rubric, and round limit; outputs are the run record and the best contract. The quality signal is binary, so iteration quality is gauged by pass count.

Automatic retirement appends a learning node: given this run's recommendations and the historical accumulated exclusion and retirement counts, aggregated per clause, stance clauses reaching the threshold are marked retired and approval status updated; the decision sits behind cross-run counting so that transient fluctuation never edits the corpus.

Tag iteration runs in two stages. The proposal stage takes the failing clauses and current tags, and a language model judges whether tags and body match, emitting a suggestion list; it only reads the clause store, hence can run concurrently. The application stage takes the human-approved changes, first acquires an advisory lock to serialize and snapshots the original tags, then writes tags and re-assembles and re-evaluates, rolling back on quality-signal degradation; tags are the retrieval axis of assembly, so the application stage must be serial and reversible.

Concurrency boundaries follow: clause-store reads run concurrently, learning-node results are consistent, and only tag application mutates the shared tags under an exclusive advisory lock, keeping corpus mutation single-threaded.

2.4 Deployment

The persistence layer is a relational database holding clauses, rubrics, evaluations, run records, and the fill cache; writes converge there and reads center on run records. Cross-entity combined queries and version comparison require strong-consistency constraints, hence the relational model.

Rendered docx and pdf results go to object storage; the database keeps only object keys, placeholder values, and digests, downloads stream back by key, and backup or distribution stays off the business query path.

Clauses and statutes additionally maintain a vector index: retrieval encodes the query with the same embedding model and recalls fragments, verifying vector dimensions before writes and requiring a rebuild on mismatch; this path is off the assembly-evaluation main chain.

An MCP server exposes assembly, evaluation, and pipelines as tools for agents and batch jobs; a web workbench provides clause review, rubric and prompt management, run dashboards, and search for human approval and inspection. Both share the service layer and the database; read-only instances mount only read endpoints, connect to read replicas, and are rate-limited.

3 Key Methods

3.1 All-Pass Binary Evaluation Protocol

The protocol takes a filled contract body and its rubric, and outputs the whole-contract quality signal plus per-criterion verdicts. Each criterion's pass standard serves directly as the yardstick of a strict binary judge; three judge models rule independently, the majority forms the verdict, and the share of judges supporting it is recorded as the confidence. Aggregation is binary: a contract passes only if every criterion passes, and fails if any criterion fails.

Weighted summation would convert a missing mandatory clause into a score deficit compensable by other clauses, letting the generator mask shortboards with strengths; binary aggregation cancels this compensation. The signal answers only whether an unsatisfied normative requirement exists, the only way for the generator to move the signal is to satisfy each requirement, and the optimization target lands on every legal point. Pass counts and per-

criterion reasoning do not enter scoring — they locate failures and track round-over-round progress. A single judge's taste cannot serve as the quality criterion: the multi-judge design addresses measurable judge biases [9] and judge-model differences large enough to flip conclusions [12], and the verdict mechanism builds on G-Eval-style evaluation [8].

3.2 Evaluation-Driven Clause-Level Self-Healing

Self-healing takes the previous round's per-criterion verdicts and confidences as input and clause-level write-back actions as output. Actions are decided by consensus confidence with a simple-majority threshold rather than unanimity: a unanimous-failure gate would leave real defects in place when two judges fail and one passes, while a single judge's dissent alone triggers nothing.

Failing items are attributed by legal-topic tag to sections, and the failure points of one section merge into a single improvement task. Write-back merges rather than replaces: an existing customized clause has its body updated, otherwise a new clause is created, because wholesale rewriting would discard what the previous round already got right while fixing one defect, and rounds would fail to converge. The loop stops on all-pass, on reaching the round limit, or when no failing item reaches the threshold; the check node keeps the historically best round by pass count.

Self-feedback iterative improvement uses the generator's own judgment as the improvement signal: Self-Refine polishes the same output from its own feedback [13], and Reflexion turns verbalized feedback into the basis of an agent's subsequent behavior [14]. The self-healing here differs in two ways. The improvement signal comes from external judges — whether to improve, which clauses, and when to stop are all decided by per-criterion verdicts, and the generator never self-evaluates. The unit of rewriting is the clause, not the full text: only the section located by the failing criterion's tag is rewritten, other sections stay untouched, and the same criteria re-check the text before and after the change.

3.3 Tag-Driven Clause Assembly

An addressable mapping between clauses and review criteria is prerequisite: otherwise a failure verdict cannot point back to a concrete clause and self-healing could only search the

full text. The corpus tags every clause with legal-topic tags drawn from contract-element-level legal points rather than section titles, and the rubric's criteria share the same identifier set; tags therefore serve both directions — self-healing locates, by a failing criterion's identifier, the clauses claiming the same point, and assembly gathers clauses by contract type with scenario and stance tags. Assembly resolves overrides with stance over scenario and scenario over master, keeping exactly one body per section, so the assembled result maps nearly one-to-one onto the criteria. Clause selection and slot filling are separated: assembly decides which legal points are addressed and outputs the placeholder body with fill instructions, filling only instantiates values, so a missing clause and a wrong value are attributable separately. Statutory-citation hardening prepares, per contract type, a block of legal-basis material used by the generator as the grounding part of the drafting prompt and by the evaluator as legal-basis criteria; the design has no controlled experiment yet, and this paper draws no validity conclusion for it.

3.4 Regression Capability of the Quality Signal

Using the quality signal as a regression baseline takes as input multiple runs of the same contract type under the same rubric, with exactly one variable under test and judge configuration and fill temperature held fixed — judge-model differences alone can flip conclusions [12], and swapping judges would change the ruler between before and after. This enables prompt A/B comparisons and no-regression checks, the former fixing type, rubric, and drafting requirements while varying only the prompt, the latter comparing pass status and per-criterion verdicts before and after a change on the same type. Both require the same rubric version on both sides, otherwise the strictness of the standard changes with the version and comparison loses meaning. The all-pass rate is expressed as the share of passing types among the 43 evaluated types; its resolution boundary is that it does not distinguish missing one criterion from missing many, pass counts compare only across rounds of the same type, and a pass-status change may also come from judge disagreement, which must be re-checked against per-criterion reasoning.

4 Experiments and Results

4.1 Setup

The evaluation set is 43 Chinese contract types — 19 typical contracts of the Civil Code contracts book and 24 atypical contracts; 23 further types do not participate, so all conclusions use 43 as denominator. The system makes no train/test split; one draft per type is scored against that type's rubric (8–11 review criteria).

Each criterion is ruled on by 3 judges independently and aggregated all-pass binary: all pass scores 1.0, any failure 0.0. Improvement triggers on FAIL_CONF_THRESHOLD — 0.99 triggers only on unanimous failure, 0.67 on majority failure (2 of 3 judges); concurrency is 12 (2 streams × 3 judges × 2 workers).

Results are taken from the run record of commit 644576e and are single-run: self-iteration spans multiple rounds but whole-loop evaluation was not repeated, so no mean or standard deviation is reported. A cross-check against the production evaluation database shows that its retained campaign state is an intermediate snapshot after round 4 (36 of 40 types had ever reached all-pass); the round-5 breakthrough types were not exported to the database, so the per-type results remain sourced from that commit's run record as authoritative.

4.2 Main Results

Of the 43 types, 39 reach all-criteria pass — 39/43 (91%), with typical contracts at 18/19 (95%) and atypical at 21/24 (88%); Table 1 gives per-type results and group subtotals, and Figure 2 the two-group distribution. The 4 failing types are lease (9/10), land transfer (10/11), film production (9/11), and service (9/11); in round 5, partnership, property service, company formation, and private equity fund turned all-pass.

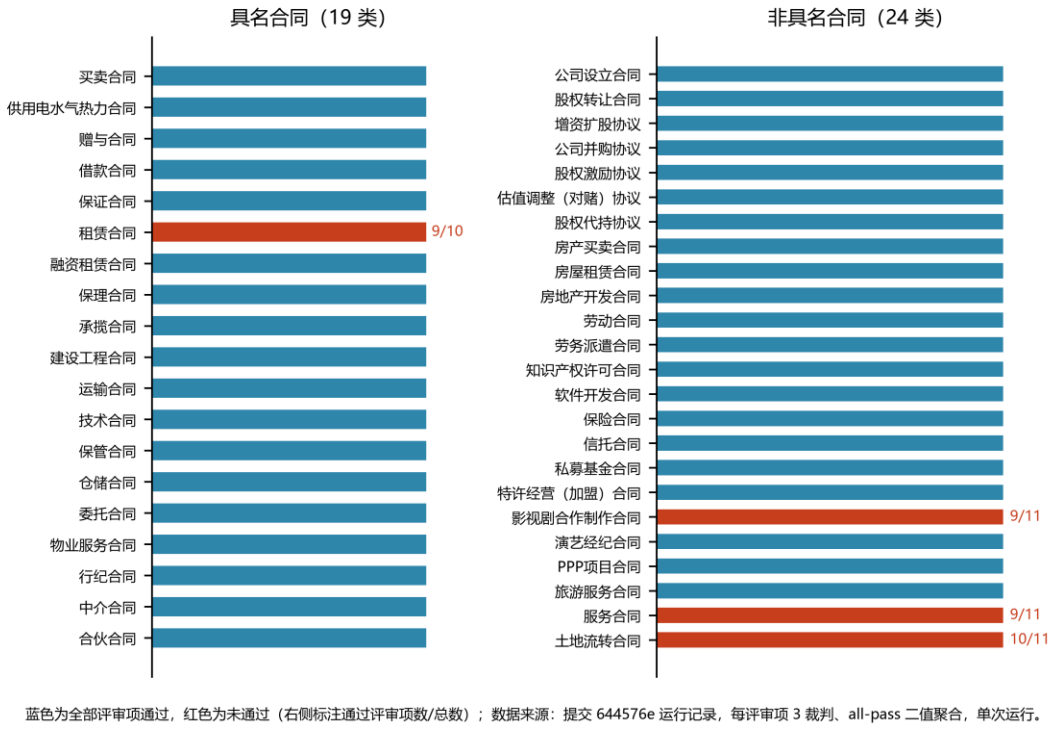


Figure 2 All-pass distribution over the 43 Chinese contract types

Under all-pass aggregation, 9/10 and 0/10 both score 0.0 — iteration quality is told by pass count, not pass rate; the same rubric and aggregation rules can re-measure any prompt or corpus change, forming a regression-comparable baseline.

Table 1 All-pass results for 43 Chinese contract types with group subtotals

Group	Contract type (type key)	All-pass	Criteria passed/total	Rounds	Pass rate
Typical	Bailment (bailment)	Yes	9/9	1	100%
Typical	Brokerage (brokerage)	Yes	—	1	100%
Typical	Construction (construction)	Yes	10/10	2	100%
Typical	Entrustment (entrustment)	Yes	10/10	1	100%
Typical	Factoring (factoring)	Yes	—	2	100%
Typical	Financing lease (financing_lease)	Yes	—	2	100%
Typical	Gift (gift)	Yes	9/9	2	100%
Typical	Guarantee (guarantee)	Yes	—	3	100%
Typical	Intermediation (intermediation)	Yes	—	1	100%
Typical	Loan (loan)	Yes	9/9	1	100%
Typical	Partnership (partnership)	Yes	10/10	5	100%
Typical	Property service (property_service)	Yes	10/10	5	100%

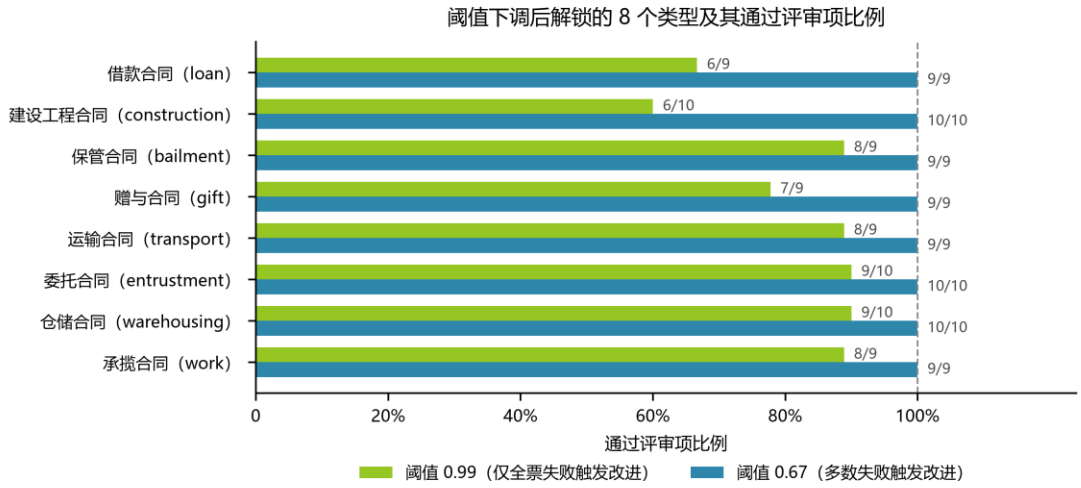
Group	Contract type (type key)	All-pass	Criteria passed/total	Rounds	Pass rate
Typical	Sale (sale)	Yes	10/10	4 (pilot run)	100%
Typical	Technology (technology)	Yes	—	2	100%
Typical	Transport (transport)	Yes	9/9	2	100%
Typical	Utilities supply (utilities_supply)	Yes	—	2	100%
Typical	Warehousing (warehousing)	Yes	10/10	1	100%
Typical	Work (work)	Yes	9/9	1	100%
Typical	Lease (lease)	No	9/10	5	0%
Atypical	Capital increase (capital_increase)	Yes	—	1	100%
Atypical	Company formation (company_formation)	Yes	9/9	1	100%
Atypical	Employment (employment)	Yes	—	1	100%
Atypical	Equity holding in trust (equity_holding_in_trust)	Yes	—	1	100%
Atypical	Equity incentive (equity_incentive)	Yes	—	3	100%
Atypical	Equity transfer (equity_transfer)	Yes	—	3	100%
Atypical	Franchise (franchise)	Yes	—	2	100%
Atypical	Insurance (insurance)	Yes	—	2	100%
Atypical	IP license (ip_license)	Yes	—	2	100%
Atypical	Labor dispatch (labor_dispatch)	Yes	—	2	100%
Atypical	Merger and acquisition (merger_acquisition)	Yes	—	2	100%
Atypical	PPP project (ppp_project)	Yes	—	2	100%
Atypical	Private equity fund (private_equity_fund)	Yes	11/11	4	100%
Atypical	Real estate development (real_estate_development)	Yes	—	2	100%
Atypical	Real estate lease (real_estate_lease)	Yes	—	1	100%
Atypical	Real estate sale (real_estate_sale)	Yes	—	2	100%
Atypical	Software development (software_development)	Yes	—	2	100%
Atypical	Talent agency (talent_agency)	Yes	—	2	100%
Atypical	Tourism service (tourism_service)	Yes	11/11	1	100%
Atypical	Trust (trust)	Yes	—	3	100%
Atypical	VAM agreement (vam_agreement)	Yes	—	3	100%
Atypical	Film production (film_production)	No	9/11	3	0%
Atypical	Land transfer (land_transfer)	No	10/11	1 (noise round)	0%

Group	Contract type (type key)	All-pass	Criteria passed/total	Rounds	Pass rate
Atypical	Service (service)	No	9/11	3	0%
Typical subtotal	19 types	18/19	—	—	95%
Atypical subtotal	24 types	21/24	—	—	88%
Total	43 types	39/43	—	—	91%

Note: data source is the run record of commit 644576e; protocol is 3 judges per criterion with all-pass binary aggregation; results are single-run records. Criterion counts of passing types that underwent no clause improvement were not exported with the record and are shown as "—"; the production-database cross-check confirms that lease, land transfer, and film production fail under both accounts, and per-criterion counts of the round-5 state remain to be exported.

4.3 Threshold Sensitivity

The 0.99 threshold reacts only to unanimous failure, and 8 types stalled on the judge-split baseline: when 2 of 3 judges report a real defect (confidence 0.67) and the third passes, aggregation yields only partial pass (e.g. loan at 6/9) and improvement never fires; after lowering the threshold to 0.67, majority failure also triggers improvement, and all 8 types turned all-pass within 2 rounds (Table 2, Figure 3).



虚线为全部评审项通过（100%）；数据来源：提交 644576e 阈值变更前后逐类记录，每评审项 3 裁判、all-pass 二值聚合，单次运行。

Figure 3 The 8 types unlocked by the threshold change and their per-criterion pass shares

The lowering works because majority failure is more tightly tied to real defects than unanimous failure: when 2 judges point at the same gap, the signal more likely corresponds to a true omission, and merge-style updating preserves what the other judges approved. The unanimous gate ties sensitivity to the lenientest judge — with any one judge passing, a real defect never triggers improvement, and repair capacity is capped by the weakest link of the measuring instrument.

Table 2 The 8 types unlocked by the threshold change (0.99 → 0.67)

Contract type (type key)	Criteria passed at 0.99	Criteria passed at 0.67	Rounds to all-pass
Loan (loan)	6/9	9/9	1
Construction (construction)	6/10	10/10	2
Bailment (bailment)	8/9	9/9	1
Gift (gift)	7/9	9/9	2
Transport (transport)	8/9	9/9	2
Entrustment (entrustment)	9/10	10/10	1
Warehousing (warehousing)	9/10	10/10	1
Work (work)	8/9	9/9	1

Note: data source is the per-type records before and after the threshold change in commit 644576e; protocol is 3 judges per criterion with all-pass binary aggregation (0.99 triggers only on unanimous failure, 0.67 on majority failure); results are single-run records.

4.4 Failure-Case Analysis

The 4 failing types each miss only one or two criteria, but the nature of the gap differs; per-criterion records separate them into two structural walls (Table 3). Lease is a clause-quality wall: the sticking point sits stably on statutory-citation accuracy, 5 rounds of rewriting never satisfied all 3 judges simultaneously — the defect is the generator's inability to self-certify the cited basis, and the fix lies in retrieval and verification of legal bases, not in more rewriting.

Land transfer, film production, and service sit at the judge-reliability ceiling: failures appear as 0.67-confidence majority splits, improvement fires but fails to converge within the round limit, and pass counts oscillate. The residual gap of these three cannot be adjudicated as

clause defects; the records show only that they are undecidable under this judge set. The blind-annotation pilot in Section 4.5 provides direct evidence — judge verdicts diverge systematically from human blind judgment — supporting the attribution of these three to the judge-reliability ceiling rather than contract defects; land transfer has only 1 round flagged as a noise round, the weakest evidence.

Partnership and property service previously stalled on a single criterion as well and turned all-pass in round 5. The two walls call for different remedies: the former recurs stably on one criterion, pointing to a generator-side capability gap addressed by legal-basis retrieval and verification; the latter appears as majority splits with cross-round oscillation, pointing to the resolution of the measuring instrument — its existence is now directly confirmed by the pilot of Section 4.5. These attributions rest on per-criterion records, confidence, and the blind pilot, not on a separated measurement of mechanism gains; the self-healing on/off ablation has not been run [to be replaced by real experiments].

Table 3 Sticking points and failure attribution of the 4 failing types

Contract type (type key)	Stuck criterion	Judge confidence	Rounds tried	Failure attribution
Lease (lease)	Statutory-citation accuracy	—	5	Clause-quality wall
Land transfer (land_transfer)	No specific criterion recorded	0.67	1 (noise round)	Judge-reliability ceiling
Film production (film_production)	No specific criterion recorded	0.67	3	Judge-reliability ceiling
Service (service)	No specific criterion recorded	0.67	3	Judge-reliability ceiling

Note: data source is the run record of commit 644576e; protocol is 3 judges per criterion with all-pass binary aggregation, confidence being the failure confidence given by the majority of judges on that criterion; types whose criterion or confidence could not be located are marked "—"; results are single-run records.

4.5 Blind Human-Annotation Pilot of Judge Validity

To give the judge-reliability attribution an independent reference, a blind human-annotation pilot was run on the production evaluation database: from all evaluation runs of the self-iteration pipeline that carry complete drafts, 3 (draft × criterion) pairs per contract type were sampled with type stratification — 126 pairs in total (42 of the 43 evaluated types have

draft-bearing runs). Each pair carries two mutually invisible annotation slots; annotators ruled blind in a web review interface — the full draft and the criterion's pass standard were visible, the judge's verdict and reasoning were not — yielding 252 pass/fail decisions.

Inter-annotator agreement is 85.7% (126 double-annotated pairs, Cohen's $\kappa = 0.522$, moderate); the two primary annotators passed 83.2% and 80.2% respectively, so their scales are close. On the same sample, judge–human divergence far exceeds human–human divergence: the judge passed only 26 pairs (20.6%), while annotators passed 206 of 252 decisions (81.7%); human–judge agreement is 36.5% (92/252), $\kappa = 0.075$. Table 5's confusion matrix shows the divergence is highly one-directional: 157 cases of human pass with judge fail against 3 in the reverse direction — the judge is systematically stricter than blind human judgment.

The pilot carries two implications. First, it supplies direct evidence for the judge-reliability-ceiling column of Table 3: with judge verdicts diverging from human blind judgment in a broad, one-directional way, the residual gap of those three "improvement-fires-but-does-not-converge" types is more plausibly attributable to the judge's standard than to contract defects. Second, it calibrates how the all-pass rate should be read: the self-healing loop takes judge all-pass as its termination signal, so 39/43 is a consistency measure under the system's internal yardstick, not a human-verified accuracy; machine self-evaluation must not be used directly as gold data, and human review must sit inside the loop — precisely this paper's design trade-off.

Validity boundaries of the pilot: the sample is type-stratified but drawn uniformly from all iterations, so early failing rounds are over-represented and the judge's fail rate on this sample (79.4%) is higher than in final-round runs; the agreement figures should be read under this sampling regime. There is one batch with mainly two annotators, no reasoning-level comparison was made, and κ is numerically unstable under skewed marginals; enlarging the sample and annotator pool and reporting per-criterion agreement remain future work.

Table 5 Confusion matrix of blind human annotation versus judge verdicts (252 decisions)

	Judge pass	**Judge fail**	**Total**
Human pass	49	157	206
Human fail	3	43	46
Total	52	200	252

Note: data source is the production-database human-review batch v4-validity-pilot-202609 (126 pairs × 2 slots, stratified 3 pairs per type, drawn uniformly from all draft-bearing iterations); annotators ruled blind (judge verdict and reasoning invisible); inter-annotator agreement 85.7% ($\kappa = 0.522$), human–judge agreement 36.5% ($\kappa = 0.075$).

4.6 Engineering Constraints

A first attempt at 36-way concurrency (4 streams × 3 judges × 3 workers) produced 0/10 for property service; investigation confirmed a rate-limiting cascade rather than a contract-quality failure: requests beyond the upstream quota return 429, and recording them directly as failures would score a contract missing only one or two criteria as zero-pass. At 12-way concurrency no 429 reappeared (Table 4).

The cascade arises because rate limits are metered per account rather than per model: the 3 judges share one limit bucket at the backend, so concurrency is not three independent quotas, and throughput is capped by the upstream account policy. Under all-pass aggregation, 0/10 and 9/10 both score 0.0 — one cascade amplifies noise into the same signal as a structural defect. Conclusions on the all-pass scale must therefore state the concurrency configuration and exclude rate limiting and transient faults from failures.

Table 4 Effect of concurrency configuration on evaluation results

Concurrency	Result	Rate-limited
4 streams × 3 judges × 3 workers = 36	Property service 0/10	Yes, 429 cascade
2 streams × 3 judges × 2 workers = 12	All types stable, no zero-pass type	No

Note: data source is the concurrency post-mortem of commit 644576e; the 0/10 at 36-way concurrency was rate-limit noise, not contract quality; results are single-run records.

5 Discussion

5.1 Effectiveness of Binarization as a Quality Signal

All-pass binarization constrains the generator's behavior, not the precision of reported scores. Weighted scoring folds criteria into mutually compensable components, so the loss of

one mandatory clause can be bought back by surplus elsewhere, letting the generator bypass single legal points to move the total; binary aggregation cancels the exchange, the signal answers only whether an unsatisfied normative requirement exists, and the only way to change the signal is to satisfy each point.

Binarization itself does not rank quality — 9/10 and 0/10 both read 0.0 — and the diagnostic duty falls to pass counts: which criterion is stuck, and whether rewriting lifts it, are answered by per-criterion records. With scale and localization thus divided, the quality signal gains both a form comparable across changes and the resolution to point back at clauses.

5.2 The Reliability Boundary of Measurement

The failing signal splits into two structural walls in the records. Lease sticks on statutory-citation accuracy alone, with rounds of rewriting merely moving the objection between judges — a clause-quality wall pointing to the generator's inability to self-certify its legal basis. Land transfer, film production, and service appear as 0.67-confidence majority splits oscillating across rounds: rewriting fixed what the objecting judges flagged yet flipped other criteria to fail — a judge-reliability ceiling; the records show only that they are undecidable under the current judge set.

Conflating the two misleads the fix in both directions: reading measurement noise as a quality defect rewrites the corpus for nothing, and attributing real defects to the judge shelves a generator-side capability gap as a tooling issue. Execution-calibrated judge evaluations likewise observe a systematic reliability ceiling [15], and meta-evaluations on domain data mark comparing judges with human judgment as a necessary condition of the method [10]; this paper's pilot provides the direct measurement on this very system — human–judge agreement of 36.5% ($\kappa = 0.075$) against inter-annotator agreement of 85.7% ($\kappa = 0.522$), with highly one-directional divergence (157 versus 3) — turning the judge-reliability ceiling from inference into measurement.

5.3 Limitations

Four boundaries limit the conclusions. Judge validity now has a first measurement from the blind pilot, but the result is negative in direction and limited in scope: human–judge

agreement is only 36.5% ($\kappa = 0.075$) and judge verdicts are systematically stricter than human blind judgment, so the 39/43 all-pass rate should be read as a consistency measure under the system's internal yardstick rather than a human-verified accuracy [10]; the pilot is a single batch with mainly two annotators and a sample skewed toward early failing rounds, and the per-criterion validity profile and stability at scale remain unmeasured. The results are single-run records: self-iteration spans rounds but whole-loop evaluation was not repeated, so 39/43 and the 8 threshold-unlocked types each constitute one observation, and multi-run stability is pending.

The gains of self-healing and of tagging have not been separated by ablation — this paper ran neither self-healing on/off nor tagged-versus-base clause comparisons, so both contributions are inferred indirectly from per-criterion records and confidence. Binary aggregation keeps only two values, and edge-quality discrimination relies on pass counts compared within a type. The evaluation set concentrates on the Chinese jurisdiction: the 43 types are drawn from the Civil Code's typical contracts and atypical contracts governed by special statutes, and there is no evidence yet that the protocol transfers to other jurisdictions or languages.

6 Conclusion and Outlook

Evaluation thereby turns from post-hoc inspection into an intrinsic driver of generation, and contract quality becomes measurable, iterable, and regression-detectable. All-pass binary aggregation cancels compensation between criteria, leaving the generator no path but to satisfy each normative requirement; threshold-triggered clause-level improvement writes back by merging rather than replacing, so already-passing content is not lost in fixing one defect; runs under the same rubric form comparable baselines in which regressions are detectable.

On 43 Chinese contract types, 39 reach all-criteria pass; the 4 failing types miss only one or two criteria, and failure does not concentrate on either the typical or the atypical side. Most types reach all-pass within the first two rounds, indicating that the default path of the clause corpus and tag assembly already covers their principal normative requirements.

Per-criterion records split the residual failures into two kinds: those whose sticking point recurs on a single criterion are clause-quality walls pointing to a generator missing self-

certifiable legal bases; those appearing as majority splits with cross-round oscillation are judge-reliability ceilings pointing to the resolution of the measuring instrument. A clause-quality wall describes unsolved-under-current-policy, not unsolvable; conflating the two misleads corpus revision and capability attribution alike.

Judge validity now has a first reference from the blind pilot: on 126 contract–criterion pairs, inter-annotator agreement is 85.7% ($\kappa = 0.522$) while human–judge agreement is only 36.5% with highly one-directional divergence — judge verdicts are systematically stricter than human blind judgment. Accordingly, the all-pass rate should be read as the internal yardstick driving self-healing, and the three failing types attributed to the judge-reliability ceiling are more plausibly limited by the judge's standard than by contract defects; the per-criterion validity profile, replication with larger annotator pools and samples, and multi-run stability reports constitute the bulk of future work.

The clause corpus and evaluation protocol are ready to be organized into an open benchmark: with types, criteria, and aggregation rules held fixed, results of different systems compare directly. Cross-jurisdiction extension requires mandatory-norm inventories, statute-retrieval sources, and corresponding criteria per target jurisdiction — the method transfers, the criterion content must be rebuilt.

Current conclusions come from single runs: multi-run means and standard deviations are pending, and the pass list holds only for this concurrency configuration. A stability report will both quantify variation under repeated evaluation and separate judge disagreement from quality fluctuation, so that a single run's gains and losses are not read as capability change; combined with the validity measurement of the blind pilot, it will convert pass rates under the internal yardstick into accuracies that can be claimed externally.

References

Format note: all entries were verified item by item on 2026-09-21 via the CrossRef API or arXiv pages; authors, titles, sources, years, and DOIs/IDs are real and traceable. Pending: volume, issue, and page numbers of journal items are to be completed from publisher pages.

[1] DAHL M, MAGESH V, et al. Large legal fictions: profiling legal hallucinations in large language models. Journal of Legal Analysis, 2024. DOI: 10.1093/jla/laae003.

[2] OGUNDE F. Legal large language models (LLMs): legal dynamos or "fancifully packaged ChatGPT"? Discover Artificial Intelligence, 2025. DOI: 10.1007/s44163-024-00167-8.

[3] GUHA N, NYARKO J, HO D E, et al. LegalBench: a collaboratively built benchmark for measuring legal reasoning in large language models. arXiv:2308.11462, 2023.

[4] LI H, CHEN Y, AI Q, et al. LexEval: a comprehensive Chinese legal benchmark for evaluating large language models. arXiv:2409.20288, 2024.

[5] YUE S, CHEN W, WANG S, et al. DISC-LawLLM: fine-tuning large language models for intelligent legal services. arXiv:2309.11325, 2023.

[6] CARNEIRO-DIAZ V, GRILLE-ZALLAS A, LAGE-ETCHART D. Automated legal analysis of rental contract clauses using large language models. SoftwareX, 2025. DOI: 10.1016/j.softx.2025.102337.

[7] DEROY A, GHOSH K, GHOSH S. Investigating legal question generation using large language models. Artificial Intelligence and Law, 2025. DOI: 10.1007/s10506-025-09452-y.

[8] LIU Y, ITER D, XU Y, et al. G-Eval: NLG evaluation using GPT-4 with better human alignment. arXiv:2303.16634, 2023.

[9] ZHENG L, CHIANG W L, SHENG Y, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685, 2023.

[10] LI Y, CHEN X, DOLATA M. LLM-as-a-Judge for mental support: a meta-evaluation using domain-specific platform data. Electronic Markets, 2026. DOI: 10.1007/s12525-026-00890-3.

[11] VASILEV Y, RAZNITSYNA I, PAMOVA A, et al. Evaluating medical text summaries using automatic evaluation metrics and LLM-as-a-Judge approach: a pilot study. Diagnostics, 2025. DOI: 10.3390/diagnostics16010003.

[12] CASSOLA-BACALLAO J, MORALES-DONAIRE J, HERNÁNDEZ-MONTOYA P, et al. Benchmarking LLM-as-a-Judge models for 5W1H extraction evaluation. Electronics, 2026. DOI: 10.3390/electronics15030659.

[13] MADAAN A, TANDON N, GUPTA P, et al. Self-Refine: iterative refinement with self-feedback. arXiv:2303.17651, 2023.

[14] SHINN N, CASSANO F, BERMAN E, et al. Reflexion: language agents with verbal reinforcement learning. arXiv:2303.11366, 2023.

[15] DEININGER P, SLANY W. Fixing, breaking, or faking it? An execution-calibrated evaluation of LLM vulnerability patching in JavaScript, Python, Go, and Java, and the limits of LLM-as-Judge. Journal of Cybersecurity and Privacy, 2026. DOI: 10.3390/jcp6050153.